دانشگاه صنعتی امیرکبیر
(پلی‌تکنیک تهران)

گزارش پروژه سوم

مبانی هوش‌مصنوعی

پرند خلیلی 40131012

پاییز 1404

۱. توصیف محیط GridWorld به عنوان یک MDP

در بخش نخست پروژه، محیط GridWorld به صورت یک فرآیند تصمیم‌گیری مارکوف (MDP) مدل‌سازی شد. مولفه‌های این MDP به شرح زیر است:

- **فضای حالت‌ها (States):** محیط شامل یک شبکه ۴×۴ است که مجموعاً ۱۶ حالت گسسته را تشکیل می‌دهد.
- **فضای اعمال (Actions):** در هر حالت، عامل امکان انتخاب ۴ عمل را دارد: بالا (Up)، راست (Right)، پایین (Down) و چپ (Left).
- **تابع انتقال (Transition):** محیط قطعی (Deterministic) است؛ یعنی با انتخاب یک عمل، عامل با احتمال ٪۱۰۰ به خانه مجاور در جهت انتخاب شده منتقل می‌شود (مگر اینکه به دیوار برخورد کند که در جای خود باقی می‌ماند).
- **پاداش‌ها (Rewards):**
  - رسیدن به هدف (حالت ۱۵ - خانه [3,3]): **+۱**
  - افتادن در تله (حالت ۵ - خانه [1,1]): **-۱**
  - هزینه حرکت (سایر جابه‌جایی‌ها): **-۰.۰۱** (برای تشویق عامل به یافتن کوتاه‌ترین مسیر).
- **حالت‌های پایانی (Terminal States):** خانه‌های شماره ۵ (تله) و ۱۵ (هدف).
- **ضریب تخفیف (Discount Factor):** مقدار gamma = 0.9 لحاظ شده است.

تحلیل تابع ارزش تحت سیاست تصادفی (Uniform Random Policy)

در ابتدا، تابع ارزش (V_π) برای یک سیاست کاملاً تصادفی (احتمال ٪۲۵ برای هر جهت) با استفاده از روش Iterative Policy Evaluation محاسبه شد.

تحلیل کیفی: نتایج نشان داد که اکثر خانه‌ها دارای ارزش منفی هستند. دلیل این امر وجود جریمه حرکت (-۰.۰۱) است. از آنجا که عامل به صورت تصادفی حرکت می‌کند، زمان زیادی را در محیط سرگردان است و پاداش‌های منفی جمع می‌کند. همچنین خانه‌های مجاورِ «تله» کمترین ارزش را دارند زیرا احتمال سقوط تصادفی در آن‌ها بالاست. خانه‌های نزدیک به «هدف» ارزش نسبتاً بهتری دارند، اما همچنان به دلیل ناکارآمدی سیاست تصادفی، مقدار آن‌ها پایین است.

۲. سیاست بهینه و تابع ارزش بهینه (Policy Iteration)

با استفاده از الگوریتم **Policy Iteration** (شامل مراحل متناوب ارزیابی سیاست و بهبود سیاست)، تابع ارزش بهینه (V^*) و سیاست بهینه (π^*) استخراج شد.

تحلیل و مقایسه با سیاست تصادفی:

۱. مقادیر ارزش: برخلاف حالت تصادفی، مقادیر V^* برای اکثر خانه‌ها مثبت شد. این نشان می‌دهد عامل یاد گرفته است مسیری را انتخاب کند که مجموع پاداش دریافتی (رسیدن به هدف منهای هزینه قدم‌ها) بیشینه شود.

۲. ساختار مسیرها: سیاست بهینه در هر خانه، جهت دقیقی را نشان می‌دهد که کوتاه‌ترین فاصله تا هدف را دارد و همزمان از خانه شماره ۵ (تله) دوری می‌کند.

۳. تفاوت: سیاست تصادفی منجر به گشت‌زنی بی‌هدف می‌شود، در حالی که سیاست بهینه یک نقشه‌ی راه قطعی برای رسیدن به هدف با کمترین هزینه است. همگرایی سریع الگوریتم (در ۲ تکرار) نشان‌دهنده کارایی برنامه‌ریزی پویا در محیط‌های کوچک و مشخص است.

۳. پیاده‌سازی Q-Learning در محیط LunarLander

در بخش دوم، مسئله کنترل فرودگر ماه که دارای فضای حالت پیوسته است، با روش **Tabular Q-Learning** حل شد.

۳-۱. فضای مشاهدات و اعمال

- **فضای مشاهدات (Observation Space)**: یک بردار ۸ بعدی پیوسته شامل: مختصات (x, y)، سرعت‌های خطی (v_x, v_y)، زاویه، سرعت زاویه‌ای و وضعیت تماس دو پایه با زمین.
- **فضای اعمال (Action Space)**: شامل ۴ عمل گسسته: (۰) هیچ‌کار، (۱) موتور چپ، (۲) موتور اصلی، (۳) موتور راست.

۳-۲. گسسته‌سازی (Discretization)

از آنجا که Q-Learning جدولی روی فضای گسسته کار می‌کند، فضای حالت پیوسته با استفاده از روش Binning گسسته‌سازی شد. برای هر یک از ۸ ویژگی حالت، بازه‌هایی تعریف شد (مثلاً ۵ یا ۶ قسمت). با استفاده از تابع np.digitize، مقادیر پیوسته سنسورها به ایندکس‌های صحیح تبدیل شدند تا بتوانیم آن‌ها را در جدول Q ذخیره کنیم.

۳-۳. توصیف الگوریتم

الگوریتم Q-Learning با فرمول به‌روزرسانی زیر پیاده‌سازی شد:

$$Q(s,a) \leftarrow (1-\alpha)\, Q(s,a) + \alpha\, [r + \gamma \max_{a'} Q(s',a')]$$

که در آن پارامترهای کلیدی به شرح زیر انتخاب شدند:

۱. **نرخ یادگیری (alpha = 0.1)**: این پارامتر تعیین می‌کند که اطلاعات جدید (پاداش دریافت‌شده و تخمین وضعیت بعدی) تا چه حد جایگزین اطلاعات قدیمی شود.

- **تحلیل:** انتخاب مقدار ۰.۱ باعث شد تا فرآیند یادگیری «پایدار» باشد. در آزمایش‌های اولیه، مقادیر بالاتر باعث نوسان شدید در جدول Q می‌شدند (عامل مدام استراتژی خود را تغییر می‌داد و همگرا نمی‌شد) و مقادیر بسیار پایین‌تر باعث می‌شدند همگرایی به ده‌ها هزار اپیزود بیشتر نیاز داشته باشد.

۲. **ضریب تنزیل (gamma = 0.99)**: این پارامتر اهمیت پاداش‌های آینده را مشخص می‌کند.

- **تحلیل:** از آنجا که هدف نهایی (فرود موفق و دریافت پاداش +۱۰۰) در انتهای اپیزود رخ می‌دهد، مقدار نزدیک به ۱ (یعنی ۰.۹۹) حیاتی است. این مقدار به عامل می‌فهماند که پایداری و حفظ تعادل در لحظات اولیه، اگرچه پاداش فوری ندارد، اما برای رسیدن به پاداش بزرگ نهایی ضروری است. مقادیر کمتر باعث «کوته‌بینی» عامل می‌شد (فقط به فکر روشن نکردن موتور برای جلوگیری از جریمه سوخت بود و در نهایت سقوط می‌کرد).

۳. استراتژی اکتشاف (epsilon-Greedy) و نرخ نزولی:

برای مدیریت چالش «اکتشاف و بهره‌برداری»، از یک استراتژی پویا استفاده شد:

- **شروع (epsilon = 1.0)**: در ابتدا عامل هیچ دانشی از محیط ندارد، بنابراین با احتمال ۱۰۰٪ حرکات تصادفی انجام می‌دهد تا فضای حالت را کاوش کند و خانه‌های جدول Q را پر کند.
- **کاهش تدریجی (Decay Rate)**: مقدار اپسیلون در هر اپیزود در عدد 0.9995 ضرب شد. این کاهش بسیار آهسته طراحی شد تا اکتشاف در طول ۵۰,۰۰۰ اپیزود ادامه یابد.
- **پایان (epsilon ≈0.01)**: در اپیزودهای پایانی، عامل تقریباً به طور کامل به دانش خود (بهره‌برداری) تکیه می‌کند و تنها ۱٪ مواقع حرکت تصادفی انجام می‌دهد تا از گیر افتادن در بهینه‌های محلی جلوگیری کند.

۴-۳. چالش‌های تنظیم پارامتر و روند بهبود

یکی از چالش‌های اصلی در پیاده‌سازی روش‌های جدولی (Tabular) برای محیط‌های پیوسته، وابستگی شدید عملکرد به «نحوه گسسته‌سازی» و «تنظیم فراپارامترها» است. در طی انجام این پروژه، نتایج اولیه با تنظیمات پیش‌فرض رضایت‌بخش نبود و عامل موفق به یادگیری فرود ایمن نمی‌شد. بهبود عملکرد طی مراحل زیر حاصل شد:

۱. بهینه‌سازی گسسته‌سازی (Discretization Bins):

در تلاش‌های اولیه، تعداد بازه‌ها (Bins) برای متغیرهای حالت کم در نظر گرفته شده بود (مثلاً تقسیم هر ویژگی به ۳ یا ۴ قسمت). این موضوع باعث می‌شد که عامل نتواند تفاوت‌های ظریف (مانند تفاوت بین سرعت کم و سرعت زیادِ منجر به سقوط) را تشخیص دهد. با افزایش دقت گسسته‌سازی (به ویژه برای زاویه و سرعت عمودی)، جدول Q بزرگتر شد اما عامل توانست کنترل دقیق‌تری روی فرودگر داشته باشد.

۲. تنظیم نرخ کاهش اپسیلون (Epsilon Decay):

در آزمایش‌های ابتدایی، نرخ کاهش اپسیلون سریع‌تر بود (مثلاً در ۵۰۰۰ اپیزود اول اپسیلون به صفر می‌رسید). این موضوع باعث می‌شد عامل قبل از اینکه تمام حالات مهم (مانند کج شدن شدید سفینه) را تجربه کند، اکتشاف را متوقف کرده و روی یک استراتژی ناقص (مانند معلق ماندن در هوا) همگرا شود. با کاهش نرخ نزولی و گسترش فرآیند اکتشاف به تمام ۵۰,۰۰۰ اپیزود، عامل فرصت کافی برای اصلاح اشتباهات خود و یافتن مسیر بهینه را پیدا کرد.

۳. تعداد اپیزودها:

با توجه به بزرگ شدن جدول Q پس از گسسته‌سازی دقیق‌تر، تعداد اپیزودهای کم (مثلاً ۱۰,۰۰۰) برای پر کردن جدول کافی نبود و بسیاری از حالات (state, action) مقداردهی نمی‌شدند. افزایش تعداد اپیزودها به ۵۰,۰۰۰ تضمین کرد که تقریباً تمام حالات محتمل، بارها دیده شده و مقدار Q آن‌ها به درستی همگرا شود.

نتیجه نهایی: با اعمال این تغییرات، میانگین پاداش از مقادیر منفی (حدود ۲۰۰- که نشان‌دهنده سقوط بود) به مقادیر مثبت و پایدار (بالای ۲۰۰+) رسید که نشان‌دهنده حل موفقیت‌آمیز مسئله است.

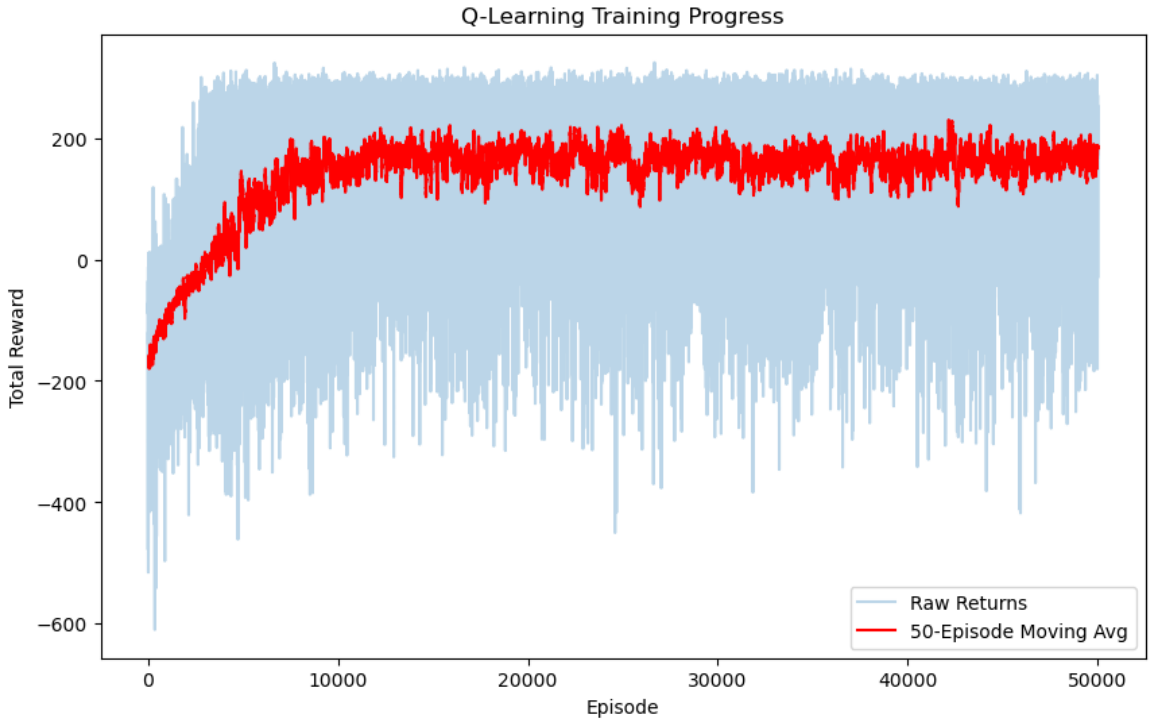
۴. تحلیل نتایج و نمودار یادگیری

۴-۱. مقایسه بازده

- **سیاست تصادفی:** میانگین بازده حدود **۱۸۰-** بود که نشان‌دهنده سقوط‌های مکرر یا خروج از صفحه است.
- **عامل آموزش دیده:** پس از ۵۰,۰۰۰ اپیزود، میانگین بازده به بالای **۲۰۰+** رسید که نشان‌دهنده حل موفقیت‌آمیز محیط است.

۴-۲. تحلیل منحنی یادگیری (Learning Curve)

نمودار زیر روند مجموع پاداش در طول ۵۰,۰۰۰ اپیزود را نشان می‌دهد:



تحلیل روند:

۱. فاز یادگیری سریع (اپیزود ۰ تا ۵,۰۰۰): نمودار با پاداش‌های منفی شروع می‌شود، اما شیب تند خط قرمز (میانگین متحرک) نشان می‌دهد عامل به سرعت قوانین فیزیک محیط و نحوه جلوگیری از سقوط سریع را یاد می‌گیرد.

۲. فاز بهبود (۵,۰۰۰ تا ۱۰,۰۰۰): پاداش‌ها وارد ناحیه مثبت می‌شوند. عامل یاد گرفته است که چگونه فرود بیاید، اما هنوز در حال بهینه‌سازی فرود نرم است.

۳. فاز همگرایی (۱۰,۰۰۰ به بعد): میانگین پاداش در بازه ۱۵۰ تا ۲۰۰ تثبیت می‌شود. ثبات نمودار در ۳۰,۰۰۰ اپیزود آخر نشان می‌دهد که الگوریتم پایدار است و دچار "فراموشی" نشده است.

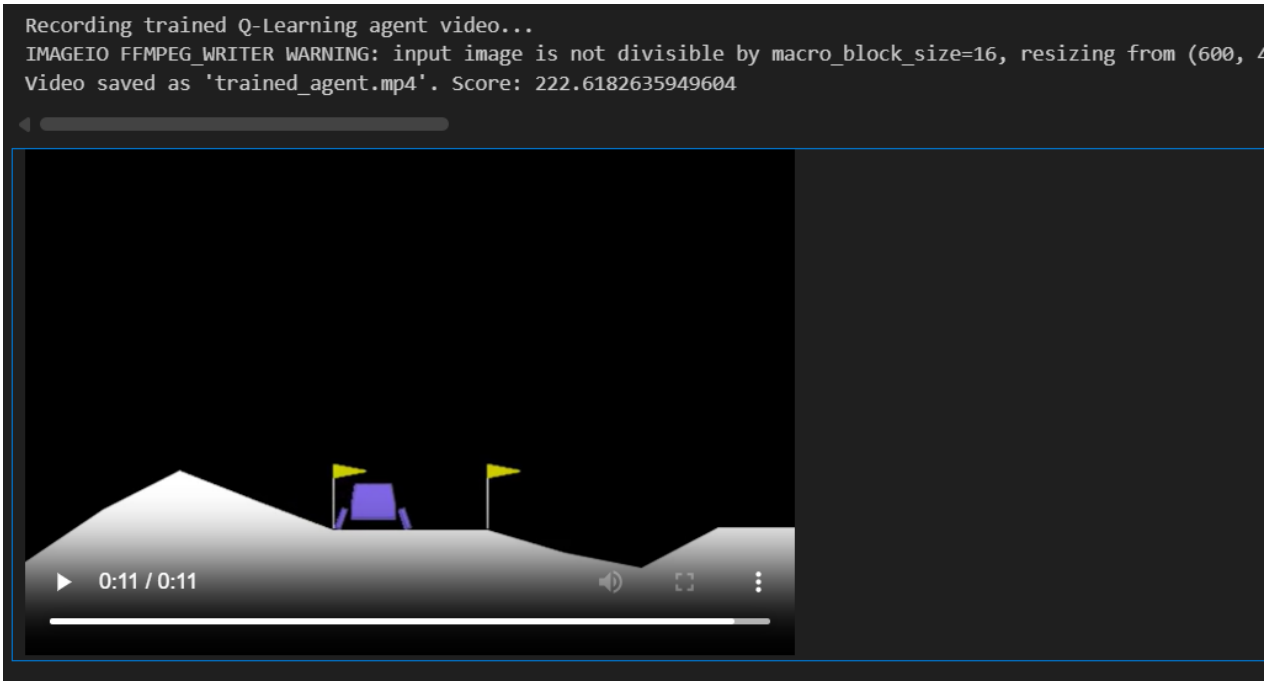
تحلیل نوسانات (Variance):

خط آبی کمرنگ (پاداش‌های خام) نوسانات زیادی دارد. این نوسانات ناشی از دو عامل است:

- **اکتشاف (Exploration):** وجود مقدار کمی epsilon باعث می‌شود عامل گاهی حرکات غیربهینه انجام دهد.
- **دشواری ذاتی محیط:** در برخی شرایط اولیه خاص (نیروی اولیه نامناسب)، فرودگر ممکن است دچار لغزش شده و پاداش کمتری بگیرد.

۳-۴. توصیف کیفی ویدیوها

- **عامل تصادفی:** رفتار کاملاً آشفته‌ای دارد. موتورها را بی‌هدف روشن می‌کند که منجر به چرخش‌های شدید و سقوط (Crash) می‌شود.
- **عامل یاد گرفته شده:** رفتار هوشمندانه‌ای دارد. به محض شروع، با موتور اصلی سرعت سقوط را کم می‌کند. از موتورهای جانبی برای حفظ تعادل و صاف نگه داشتن زاویه استفاده می‌کند و نهایتاً با سرعت کم و دقیقاً بین پرچم‌ها فرود می‌آید (Soft Landing).



۵. بحث و بهبودهای ممکن

محدودیت‌های روش Q-Learning جدولی

۱. مشکل ابعاد (Curse of Dimensionality): با گسسته‌سازی، اندازه جدول Q به شدت بزرگ می‌شود. پر کردن تمام خانه‌های این جدول نیازمند تعداد بسیار زیادی اپیزود است (همانطور که ۵۰,۰۰۰ اپیزود نیاز بود).

۲. از دست دادن دقت: گسسته‌سازی باعث می‌شود تفاوت‌های جزئی در سرعت یا موقعیت نادیده گرفته شوند که مانع از کنترل بسیار دقیق و ظریف (Fine-tuning) می‌شود.

ایده‌هایی برای بهبود

۱. استفاده از Deep Q-Network (DQN): جایگزینی جدول Q با یک شبکه عصبی (Neural Network) اجازه می‌دهد عامل مستقیماً با ورودی‌های پیوسته کار کند و قدرت تعمیم‌پذیری (Generalization) بسیار بالاتری داشته باشد.

۲. گسسته‌سازی تطبیقی (Adaptive Discretization): استفاده از سبدهای ریزتر در نواحی حساس (نزدیک زمین و سرعت صفر) و سبدهای درشت‌تر در نواحی دورتر.

۳. استراتژی اکتشاف پیشرفته: استفاده از روش‌هایی مانند Softmax Action Selection یا UCB به جای epsilon-Greedy ساده برای مدیریت بهتر اکتشاف.